

L8 Principal 的 Agentic Engineering Workflow: 生产级 Agent 工程 workflow

视频讲者: Kun; 结构化笔记整理: GPT-5.5 Pro

2026 年 6 月 25 日



视频作者/频道: Kun (字幕自述; 本地视频文件未包含频道元数据)

发布日期: 未知 (本地文件未包含发布日期元数据)

视频时长: 00:45:45.96

视频链接: <https://www.youtube.com/watch?v=iQyg-KypKAA>

处理来源: 本地 webm 视频与两份 vtt 字幕。官方封面未随文件提供, 首页使用 00:00:00 本地视频画面作为封面。

PDF 制作 Skill: [wdkns/youtube-render-pdf](https://github.com/wdkns/youtube-render-pdf)

目录

1 课程地图：从单人写代码到船长式协作	2
1.1 视频要解决的问题	2
1.2 术语对照	3
1.3 一个可计算的吞吐量模型	3
1.4 本章小结	4
2 搭船：终端、tmux 与 Neovim 组成控制台	4
2.1 为什么从终端开始	4
2.2 WezTerm：跨平台与可脚本化配置	4
2.3 tmux：把 agent 会话变成持久工作站	5
2.4 Neovim：让编辑动作保持在键盘路径上	5
2.5 本章小结	6
3 招募船员：agent harness、memory file 与 skill	6
3.1 四类 agent harness 的选择	6
3.2 全局 memory file：只放跨项目偏好	7
3.3 项目 memory file：把项目经验沉淀给后续会话	8
3.4 Skill：把条件性长知识移出 memory file	9
3.5 本章小结	10
4 与单个船员协作：语音、工具接口与可视化规划	10
4.1 语音输入：降低提示词输入摩擦	10
4.2 工具接口：token、轮次、延迟都是成本	11
4.3 Lavish：把需求讨论从文字墙变成可标注 artifact	12
4.4 本章小结	15
5 质量门禁：No Mistakes 把首版代码送到干净 PR	15
5.1 为什么不能把 diff review 当作主路径	15
5.2 No Mistakes 的管线步骤	15
5.3 人类时间集中在开头和结尾	16
5.4 本章小结	17
6 延长无人值守时间：Goodnight Have Fun	17
6.1 为什么需要长循环	17
6.2 适合长循环的目标类型	17
6.3 本章小结	18
7 多船员并行：git worktree 与 Treehouse	18
7.1 并行时的文件系统冲突	18
7.2 Treehouse：管理 worktree 的轻量层	19
7.3 本章小结	20

8 大副接管调度：First Mate	20
8.1 从直接管理每个 agent 到管理一个调度者	20
8.2 Open Super Whisper 的初始 prompt	20
8.3 First Mate 的实际调度过程	21
8.4 瓶颈转移：从执行不足到方向不足	22
8.5 本章小结	22
9 实践清单：把视频 workflow 落地到自己的项目	22
9.1 第一周：先搭一个可恢复的控制台	22
9.2 第二周：建立记忆与 skill 纪律	23
9.3 第三周：把需求和验证管线前后夹住	23
9.4 第四周：开始并行与调度	23
9.5 本章小结	23
10 总结与延伸	23
10.1 讲者的收束观点	23
10.2 本笔记的压缩结论	24
10.3 下一步问题	24
10.4 本章小结	24

1 课程地图：从单人写代码到船长式协作

1.1 视频要解决的问题

这节课围绕一个非常具体的工程问题展开：当 coding agent 已经能快速写出大量代码之后，专业工程师怎样避免被提示词、diff、测试、PR、并行任务和上下文切换淹没。讲者 Kun 把自己放在“船长”的角色中：终端和工具链是船，agent 是船员，First Mate 是大副，人的职责逐步上移到方向选择、需求澄清、质量门禁和用户洞察。

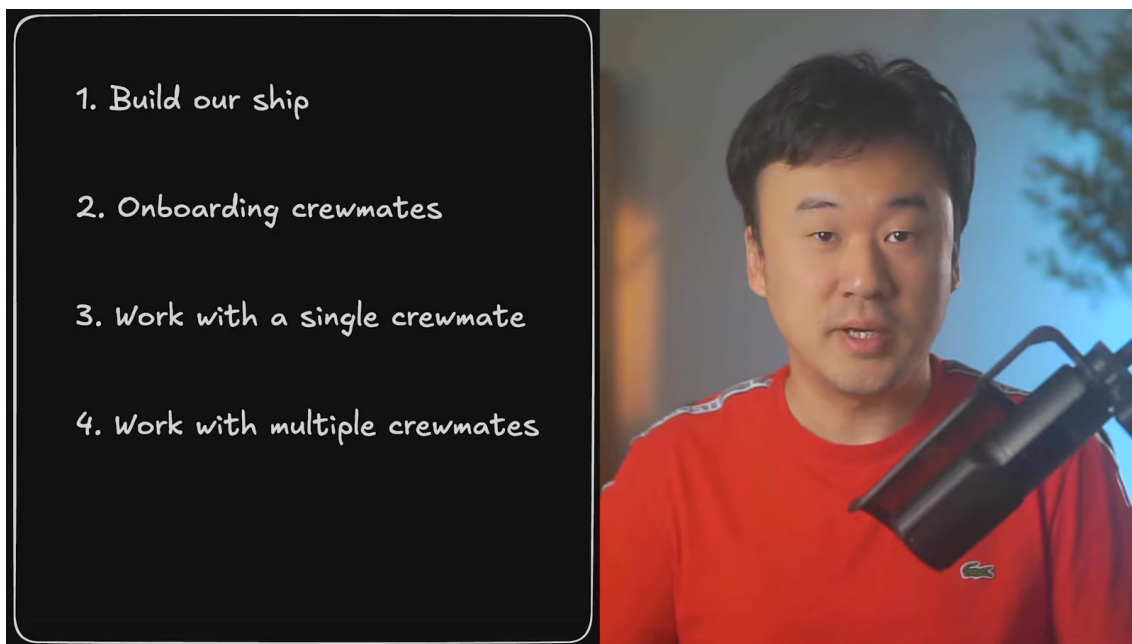


图 1: 课程的五段路线：搭船、招募船员、单船员协作、多船员并行、大副接管管理开销。¹

全课可以压缩成一条主线：先把个人开发环境改造成可持续调度 agent 的控制台，再用 memory 和 skills 为 agent 建立入职制度，然后用 Lavish 做需求规划、用 No Mistakes 做质量门禁、用 Goodnight Have Fun 延长无人值守执行时间、用 Treehouse 隔离并行 worktree，最后让 First Mate 把这些工具串成统一的任务调度层。

核心心智模型

人类工程师的高价值位置逐渐从“直接写每一行代码”移动到四个动作：定义目标、压缩需求歧义、设置验证管线、判断下一步方向。agent 负责中间的大量实现工作，工具链负责把实现工作装进可审计、可回滚、可并行的流程里。

¹ 视频画面时间区间：00:01:46–00:02:08。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

1.2 术语对照

术语	在本课中的具体含义
Agent harness	承载 coding agent 的交互外壳，例如 Claude Code、Codex CLI、Pi、Open-Code。它决定默认体验、模型接入、权限、TUI、可定制性。
Memory file	每次 agent 会话自动加载的 Markdown 记忆。全局记忆保存个人偏好，项目记忆保存 repo 结构、术语、测试方式和项目习惯。
Skill	条件触发的操作说明包。启动时只暴露短描述，agent 需要时再读取完整内容，适合把“偶尔才需要”的长说明移出 memory file。
AXI	Agent eXperience Interface。讲者提出的一组面向 agent 的工具设计标准，目标是减少 token、轮次和等待时间。
Lavish	一个用于复杂需求规划的 AXI 工具，让 agent 生成和项目设计系统一致的 HTML artifact，支持视觉讨论、局部标注和决策按钮。
No Mistakes	把 agent 的首版实现推进到干净 PR 的验证管线，包括隔离 worktree、rebase、对抗审查、端到端测试、证据记录、文档和 lint。
Goodnight Have Fun	长时间循环执行工具。适合可验证目标或人类信任 agent 判断的目标，例如页面加载时间、E2E 覆盖率、可用性问题的搜寻。
Treehouse	管理 git worktree 的工具。它创建、复用、标记闲置 worktree，降低多 agent 并行时的人脑管理负担。
First Mate	任务调度层。人对它下达项目级目标，它拆成多个并行任务，调用 Treehouse、agent 和 No Mistakes，等待人类做方向与最终判断。

1.3 一个可计算的吞吐量模型

视频中反复出现的工程判断可以用一个简化模型表达。有效吞吐量取决于并行任务数量、任务通过管线的概率、结果贴合真实需求的程度，以及人类在需求、审核、切换上的时间成本：

$$\text{有效吞吐量} = \frac{N \cdot p_{\text{pass}} \cdot q_{\text{fit}}}{T_{\text{spec}} + T_{\text{review}} + T_{\text{switch}}}$$

- N ：同时推进的任务数。Treehouse 和 First Mate 的价值主要体现在扩大这个数。
- p_{pass} ：任务通过验证管线的概率。No Mistakes 通过对抗审查、E2E 和 CI babysitting 提升它。
- q_{fit} ：结果与真实产品目标的贴合度。Lavish 通过视觉 artifact 和明确决策提升它。
- T_{spec} ：人类澄清需求的时间。语音输入和 Lavish 会影响它。
- T_{review} ：人类审核最终结果的时间。证据截图、风险评估和干净 PR 会影响它。
- T_{switch} ：上下文切换损耗。tmux、状态栏、Treehouse 和 First Mate 都在压低它。

这个式子解释了一个容易忽略的点：只追求模型写代码速度会很快撞到人类审核瓶颈。Kun 的整套 workflow 逐段减少的是分母中的人工摩擦，同时用验证管线保护分子中的质量。

1.4 本章小结

本课的主题聚焦于个人工程系统的重构：把需求、计划、实现、验证、PR 与并行调度串成一条生产线，而非停留在“把某个 agent 用得更熟”。后续章节会沿着这条线逐层展开。

2 搭船：终端、tmux 与 Neovim 组成控制台

2.1 为什么从终端开始

Kun 的第一层选择是把几乎所有工作放在终端里。动机有两个。第一，双手停留在键盘上时，思路被鼠标打断的次数会减少；这对 agent workflow 很重要，因为工程师经常需要在提示词、日志、测试、编辑器和多个会话之间快速移动。第二，同一套终端流程可以复用在 Mac、Linux、Windows，甚至可通过远程会话延伸到手机。

相关概念：控制台的三层结构

终端模拟器负责显示和输入，例如 WezTerm；terminal multiplexer 负责把一个终端拆成多个持久 pane/window，例如 tmux；编辑器和 agent harness 运行在这些 pane/window 里，例如 Neovim、Claude Code、Codex CLI、Pi、OpenCode。三层合在一起，才形成可并行调度 agent 的控制台。

2.2 WezTerm：跨平台与可脚本化配置

WezTerm 在视频中承担“船体外壳”的角色。讲者强调两点：它跨平台，且配置文件是 Lua 脚本。Lua 配置可以写条件和逻辑，颜色主题、快捷键和行为能热加载。视频里把主题切换到 Chalk，再切回 Rose Pine Moon，演示的是配置修改后立即生效。

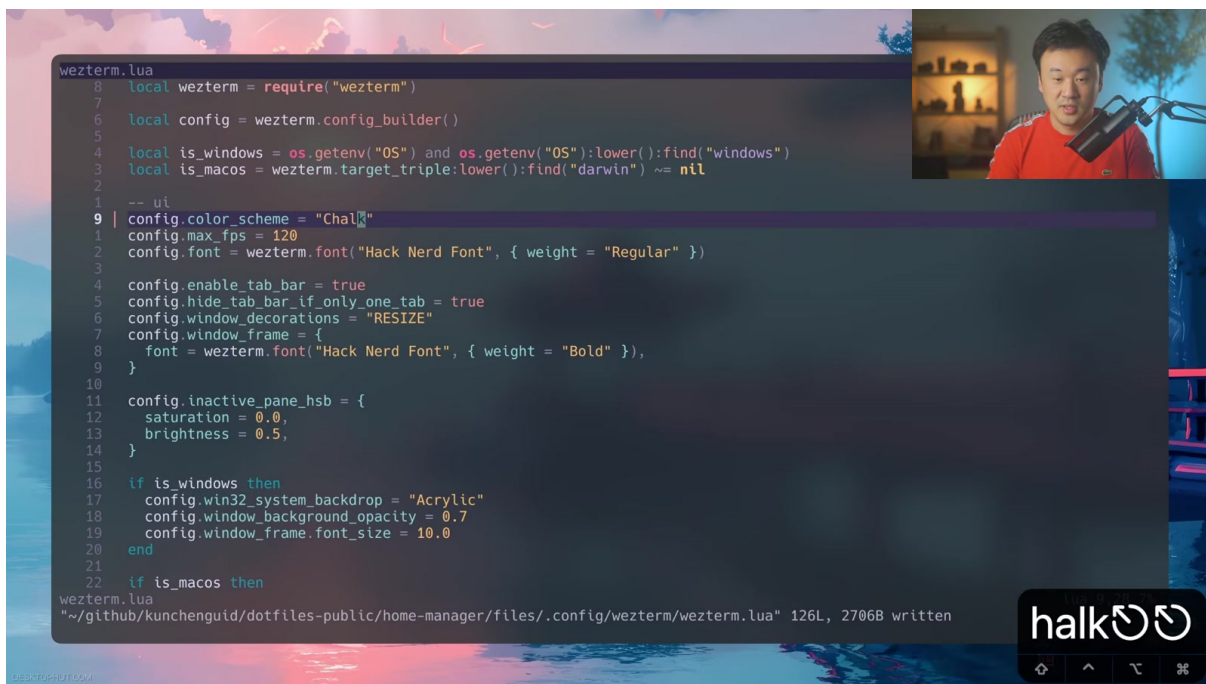


图 2: WezTerm 的 Lua 配置：终端主题、键位和行为通过脚本统一管理，并支持热加载。²

²视频画面时间区间：00:04:54-00:05:08。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

2.3 tmux: 把 agent 会话变成持久工作站

tmux 的价值来自三个细节。第一, pane 可以把编辑器、agent、命令行放在同一个屏幕里。第二, window 可以承载多个任务上下文。第三, tmux session 持久存在于服务器进程里; detach 后终端关闭, attach 后恢复到原状态。这让一台机器上的 agent 任务可以从另一台笔记本或手机继续观察。

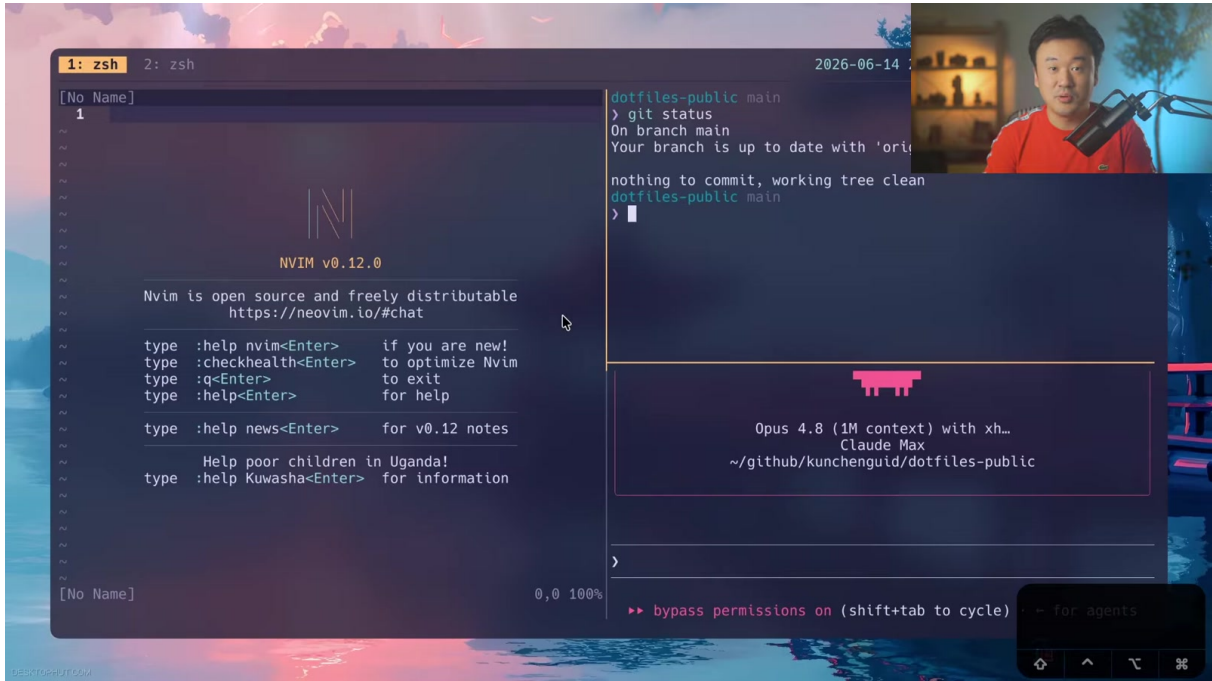


图 3: tmux pane 与 window: 一个 pane 启动 agent, 一个 pane 放编辑器, 一个 pane 留给命令。³

```
1 # start or attach a persistent session
2 tmux new -A -s agents
3
4 # inside tmux: split panes and create windows with key bindings
5 # detach: leave the session running in the background
6 # attach again from laptop or phone: return to the same state
```

Listing 1: 视频中 tmux 用法的抽象命令形态

2.4 Neovim: 让编辑动作保持在键盘路径上

Neovim 在 workflow 中承担低摩擦编辑器的角色。视频演示了 Vim 模式、相对行号、dd 删除当前行、u 撤销、11k 跳到上方第 11 行、space s 全库搜索、space f 文件名检索。关键点不在这些按键本身, 而在一组可重复肌肉记忆: 当 agent 产生 diff 或需要人工微调时, 人类能够快速定位、观察、改动和返回 agent 会话。

³ 视频画面时间区间: 00:06:20-00:06:42。若画面经过缩放或裁剪, 时间区间仍对应原始本地视频。

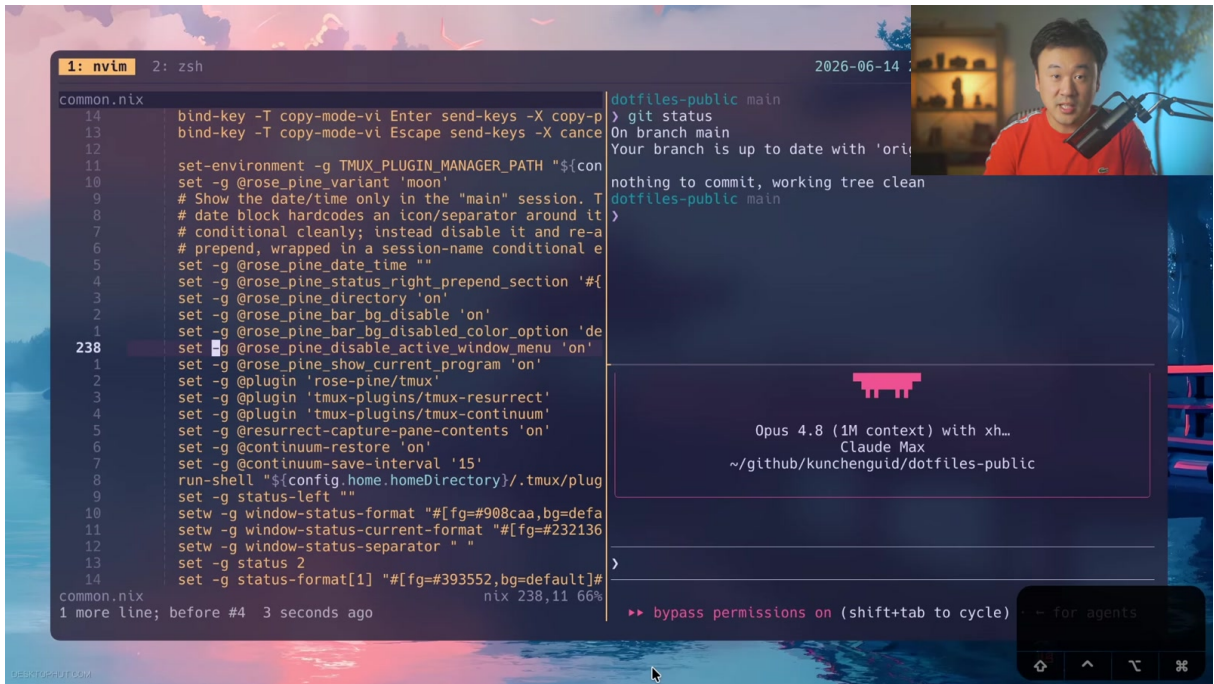


图 4: Neovim 的相对行号与键盘移动：讲者用 11k 等动作展示无鼠标导航。⁴

常见误区：终端审美和生产能力是两件事

漂亮的终端主题本身不能提升 agent 成功率。视频中真正影响生产力的是持久会话、键盘优先、pane/window 编排、跨设备恢复和低切换成本。缺少这些机制时，终端只会变成另一个需要手动管理的窗口。

2.5 本章小结

搭船阶段的目标是让人类工程师拥有一个稳定控制台：agent 可以长时间运行，任务可以分屏观察，编辑器可以快速介入，远程设备可以恢复上下文。这为后续的 memory、skill、验证管线和并行工作打地基。

3 招募船员：agent harness、memory file 与 skill

3.1 四类 agent harness 的选择

视频提到四个常用 harness。Claude Code 的默认体验最完整，适合已经在 Anthropic 订阅体系中的用户；Codex CLI 用 Rust 编写，视频中感受更顺滑，且开源便于遇到问题时让 Codex 自己检查源码；Pi 走极简和高扩展路线，适合愿意自己改造的人；OpenCode 的 TUI 更成熟，模型接入范围广，开箱功能比 Pi 更丰富。讲者后续用 Claude Code 演示，因为观众熟悉度更高；整套设计保持 agent agnostic，避免绑定某一个模型或外壳。

⁴视频画面时间区间：00:07:48-00:08:08。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

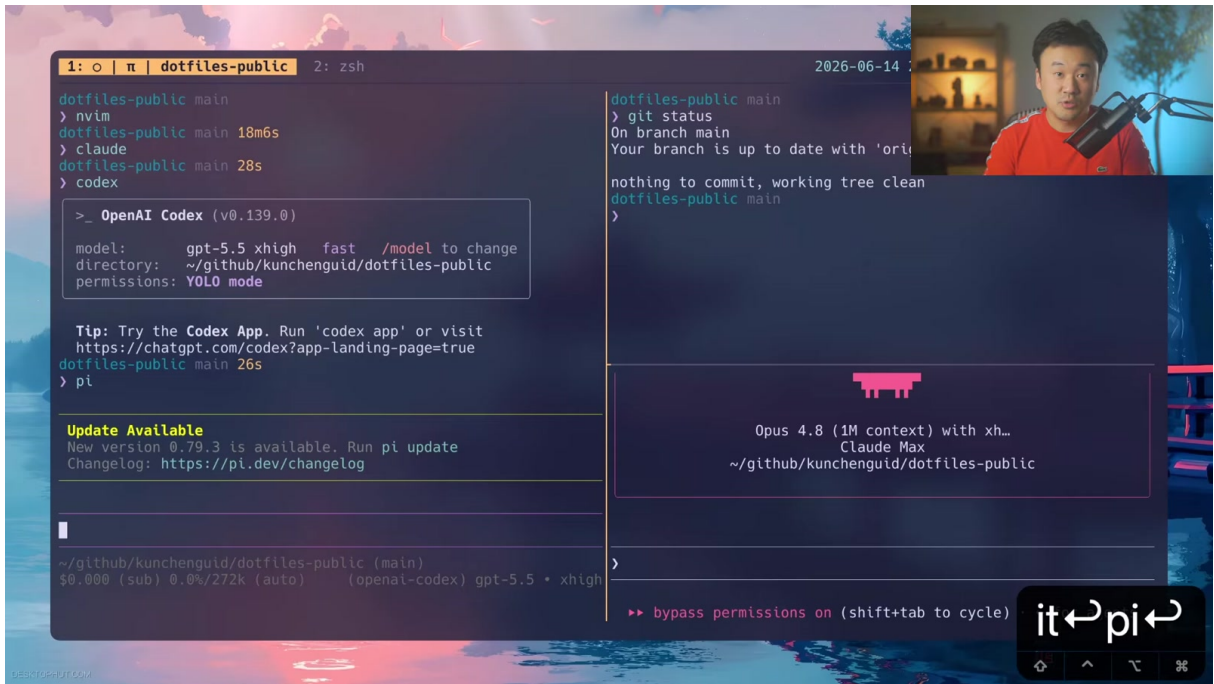


图 5: 同一 tmux 工作区里启动 Claude Code、Codex CLI、Pi 等 harness, 展示 workflow 对 harness 的低耦合。⁵

3.2 全局 memory file: 只放跨项目偏好

全局 memory file 会进入每个 agent session 的系统提示。Kun 的实际全局文件只有 27 行, 说明它更像“船规”, 而非资料库。视频给了三个例子: 写 PR 描述时避免 em dash; 做技术决策时降低“开发成本”的权重; 修 bug 时先在最接近终端用户体验的 E2E 场景中复现。

⁵ 视频画面时间区间: 00:10:18–00:10:42。若画面经过缩放或裁剪, 时间区间仍对应原始本地视频。

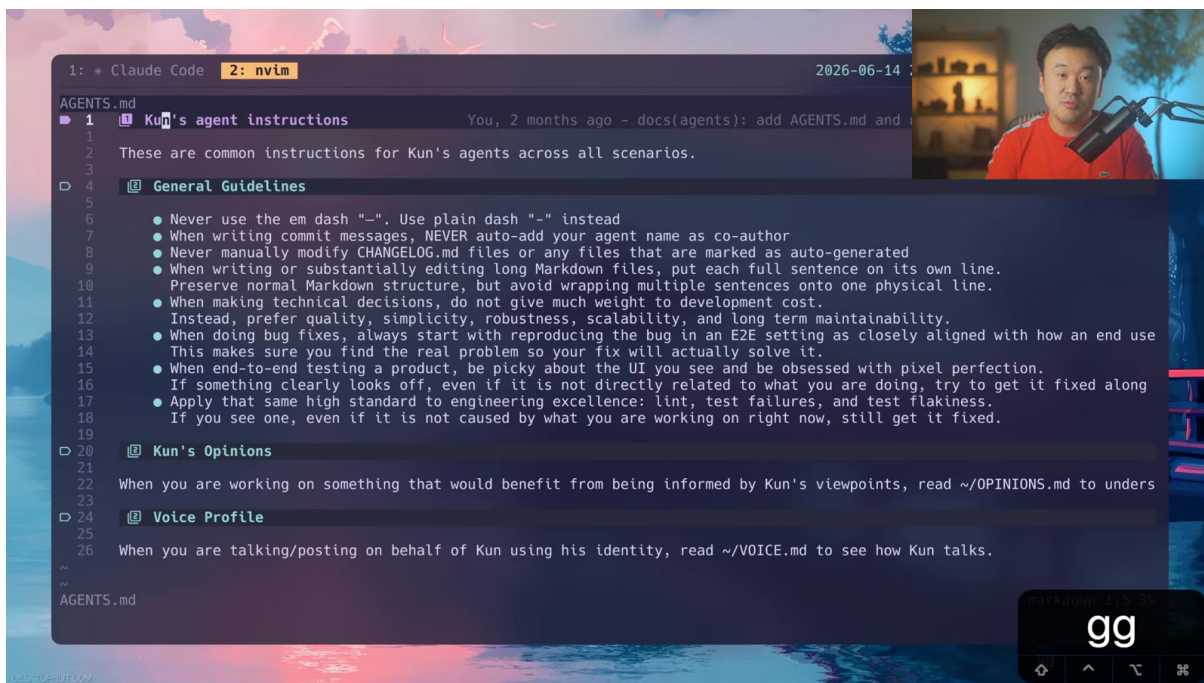


图 6: 全局 memory file 的示例：个人偏好和跨项目原则保持短小，避免每次会话都消耗过多 token。⁶

为什么“开发成本”需要校正

模型从大量人类工程数据中学会估算项目工期，常会把 AI 自己几分钟能完成的原型估成“几天、几周、几个月”。如果不校正这个偏差，模型会偏向短期便宜方案，牺牲可维护性、可扩展性和产品完整度。

3.3 项目 memory file：把项目经验沉淀给后续会话

项目级 memory file 保存 repo 层面的知识：项目是什么、目录怎样组织、关键术语怎样使用、核心组件怎样协作、E2E 测试怎样跑、代码约定有哪些。它来自反复纠错：每当 agent 犯了可重复的错，人类让它把经验写回 memory file。随着项目推进，这份文件变成“船员在同一艘船上积累的航海日志”。

⁶ 视频画面时间区间：00:12:24-00:12:48。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

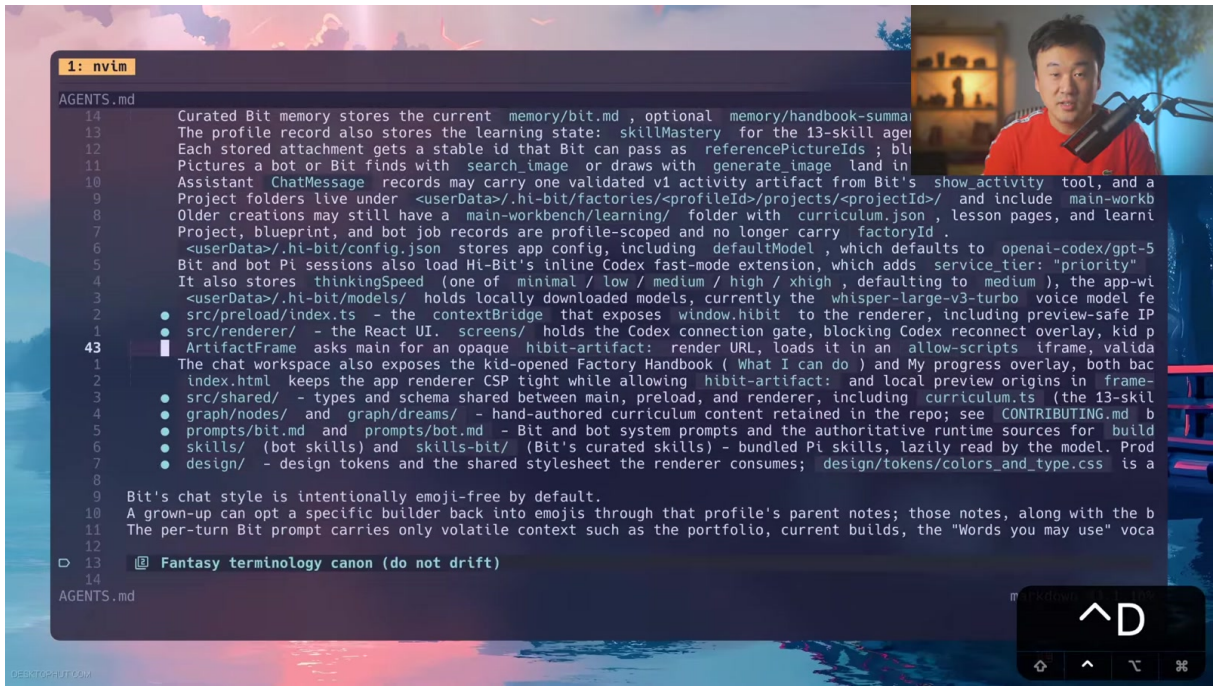


图 7: 项目 memory file: 包含项目背景、repo 结构、术语、组件机制、E2E 测试和约定。⁷

3.4 Skill: 把条件性长知识移出 memory file

项目记忆会膨胀。讲者的做法是把只在特定场景才需要的内容改成 skill。视频示例是把 E2E testing instructions 从 AGENTS.md 中抽出，生成项目级 skill。agent 启动时只加载 skill 的短描述；当它决定要做 E2E 测试时，再读取完整说明。这种 progressive disclosure 节省 token，也让 memory file 更短。

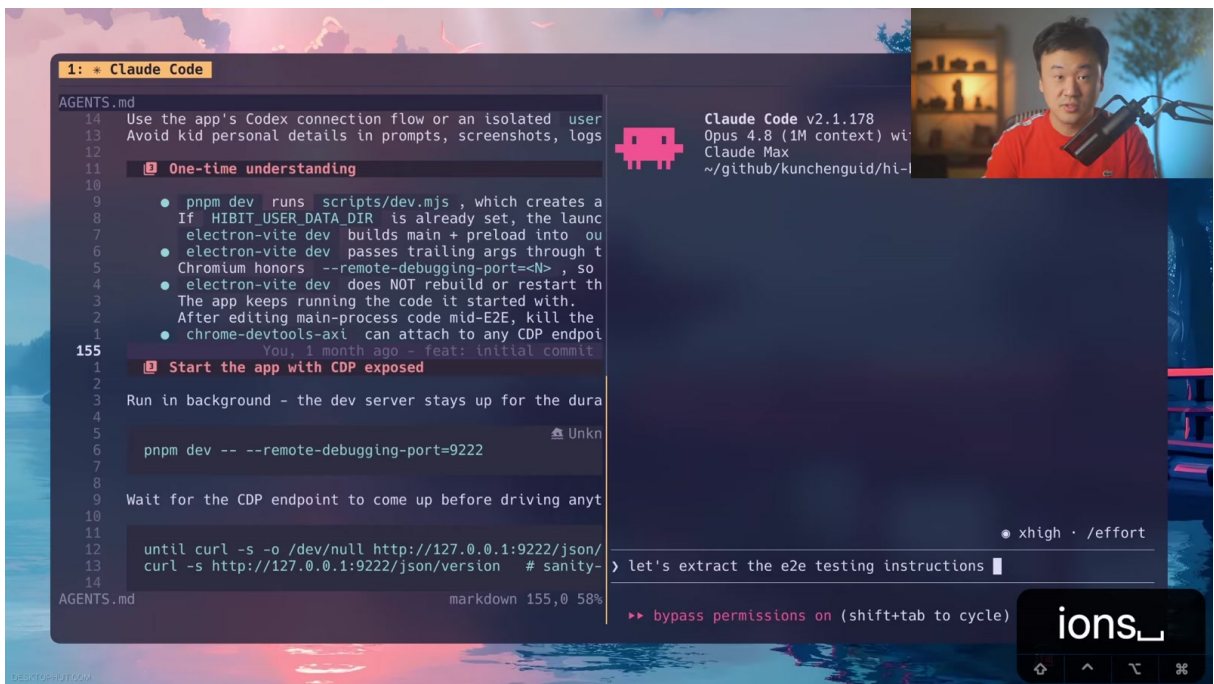


图 8: 将 E2E testing instructions 从项目记忆抽到项目级 skill，减少默认上下文。⁸

⁷ 视频画面时间区间: 00:15:22-00:15:46。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

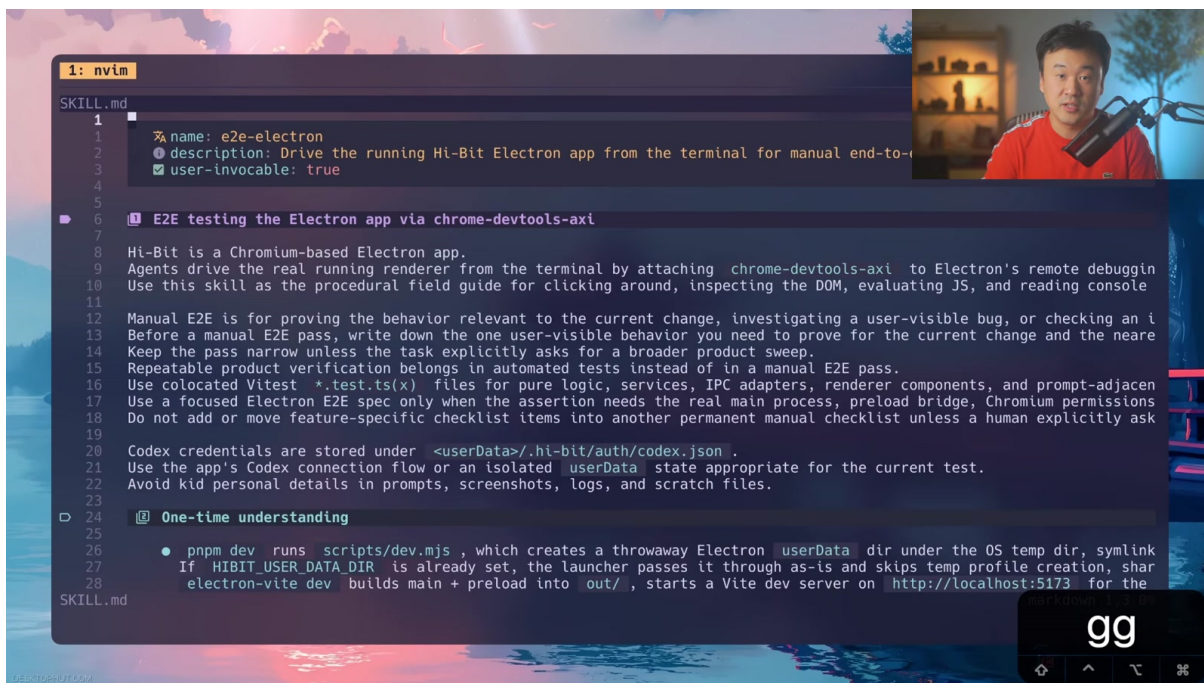


图 9: Skill 文件中的短描述与长说明：启动时暴露描述，需要时读取全文。⁹

安全边界：skill 会指挥 agent 在本机执行动作

从互联网安装 skill 时要关注权限和供应链风险。skill 可以让 agent 运行命令、读取文件、调用网络工具。视频提醒：GitHub stars 代表传播热度，不能直接代表安全性或有效性；讲者还提到一个高星 skill 在自己的 Program Bench 评估中让 token 增加约 5%，结果变差。

3.5 本章小结

招募船员阶段的关键是“少量稳定全局规则、项目级经验积累、按需展开的 skill”。这三者分别对应个人偏好、项目知识、条件性操作手册。缺少这三层，agent 每次上船都像新员工第一天入职。

4 与单个船员协作：语音、工具接口与可视化规划

4.1 语音输入：降低提示词输入摩擦

Kun 已经把大部分 prompt 改成语音输入。视频引用一篇关于 speech and keyboard text entry 的论文，说明说话可显著快于打字；他在实际 workflow 中用 Open Super Whisper 做本地转写。需要 URL、文件路径等精确字符串时，才回到键盘输入。这个选择让人类更愿意给 agent 充分背景，而不被长 prompt 的打字成本卡住。

⁸视频画面时间区间：00:16:54-00:17:12。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

⁹视频画面时间区间：00:18:28-00:18:48。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

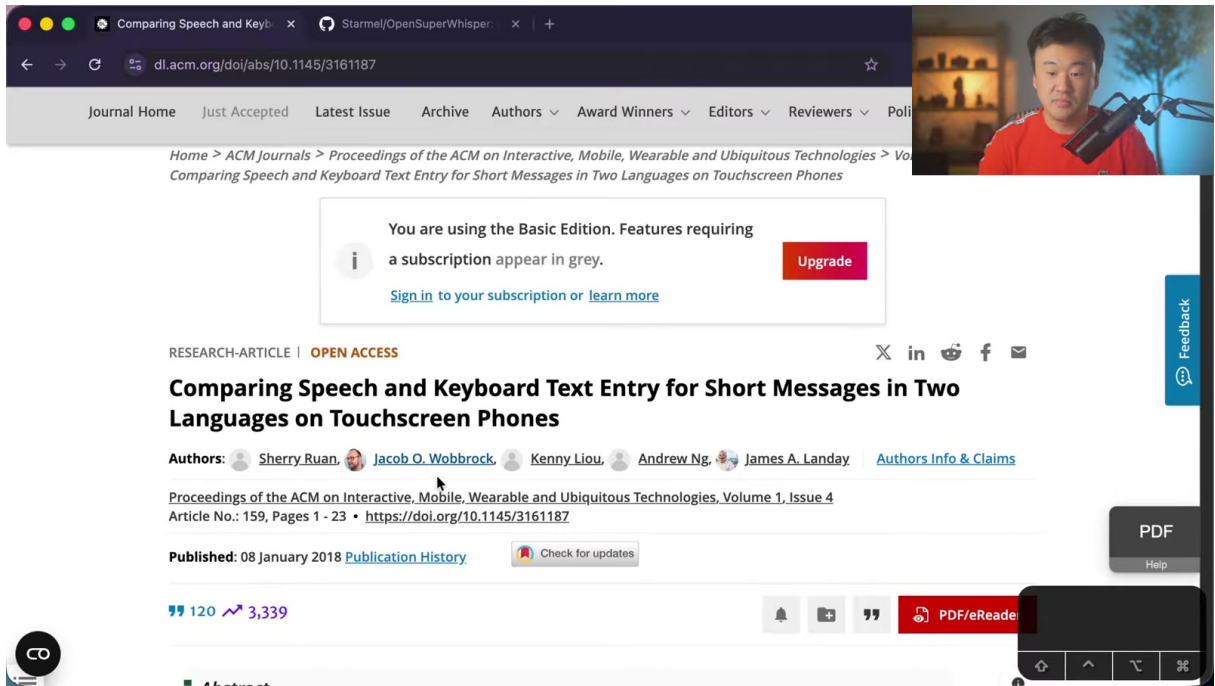


图 10: 视频展示的论文页面: 讲者用它说明语音输入相对键盘输入的效率优势。¹⁰

4.2 工具接口: token、轮次、延迟都是成本

当 agent 需要查 PR、GitHub issue、浏览器状态或测试结果时,它依赖外部工具。视频中对 GitHub 访问方式做了基准: AXI 条件在 85 次运行中达成 100% success, 平均成本约 \$0.050, 平均时长 15.7 秒, 平均 3 轮; CLI 条件 success 为 86%, 平均成本约 \$0.054, 平均时长 17.4 秒, 平均 3 轮; 几个 GitHub MCP 条件的成本、时长和轮次更高。讲者据此强调: 工具输出格式、工具搜索方式和接口粒度会直接影响 agent 的成本与成功率。

¹⁰ 视频画面时间区间: 00:20:56-00:21:18。若画面经过缩放或裁剪, 时间区间仍对应原始本地视频。

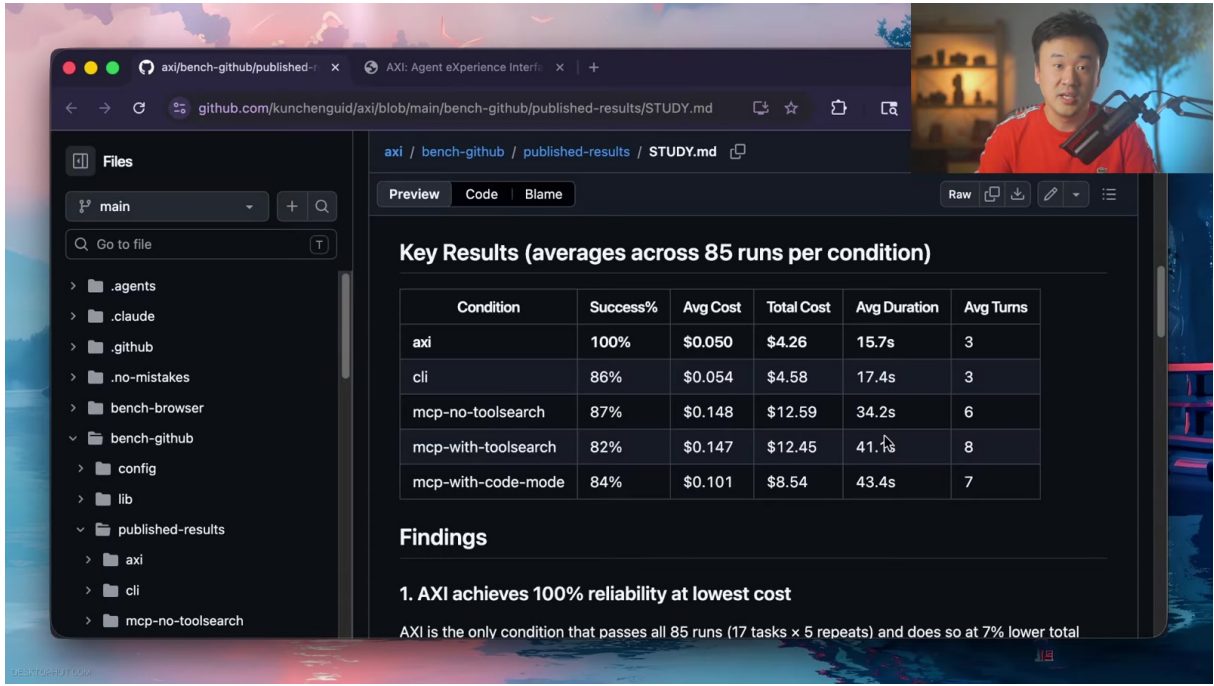


图 11: GitHub 访问方式基准: AXI 在视频展示的表格中取得最低成本和最高成功率。¹¹

这可以用一个简单的成本式表示:

$$C_{\text{run}} = c_{\text{tok}} \cdot (T_{\text{prompt}} + T_{\text{tool}} + T_{\text{reason}}) + c_{\text{latency}} \cdot R$$

- C_{run} : 一次 agent 任务的综合成本。
- c_{tok} : 单位 token 的价格或预算权重。
- T_{prompt} : 系统提示、memory、skill 描述和用户 prompt 的 token 数。
- T_{tool} : 工具返回内容的 token 数, JSON、Markdown 表格、紧凑文本格式会影响它。
- T_{reason} : 模型中间推理和回复消耗的 token 数。
- c_{latency} : 等待时间的业务权重。
- R : 工具调用和 agent 交互轮次。

AXI 的重点在于把工具当作 agent 的第一等用户来设计。人类 GUI 追求可点击、可读、美观; agent 工具接口更关注可压缩、可解析、少轮次、少歧义、低权限摩擦。

4.3 Lavish: 把需求讨论从文字墙变成可标注 artifact

默认 planning 往往给出一大段 Markdown 或终端文本。视频中, Kun 让 agent 为 High Bit 的 “what I can do” 和 “my progress” 两个按钮设计合并方案。默认模式需要人阅读一整堵文本, 难以指出某个局部方案。Lavish 的做法是让 agent 生成 HTML artifact, 使用当前项目的 design system 展示方案, 并在页面上提供标注、评论和决策按钮。

¹¹ 视频画面时间区间: 00:22:54-00:23:12。若画面经过缩放或裁剪, 时间区间仍对应原始本地视频。

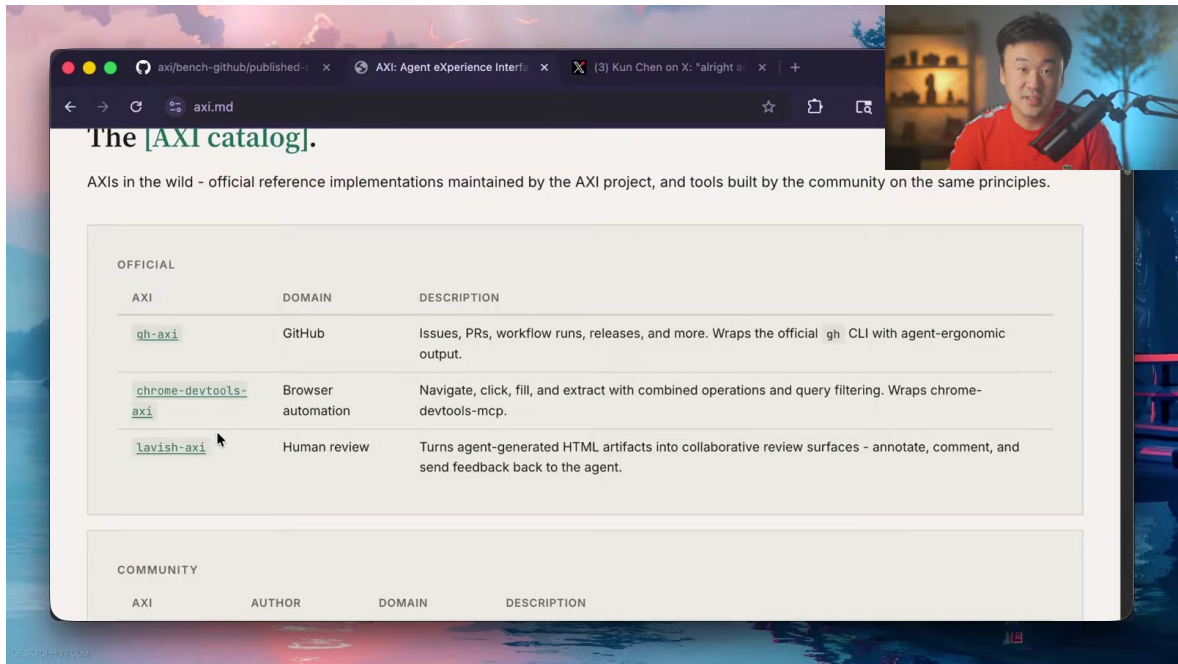


图 12: AXI catalog 中的 Lavish AXI: 讲者把它作为复杂功能规划的核心工具。¹²

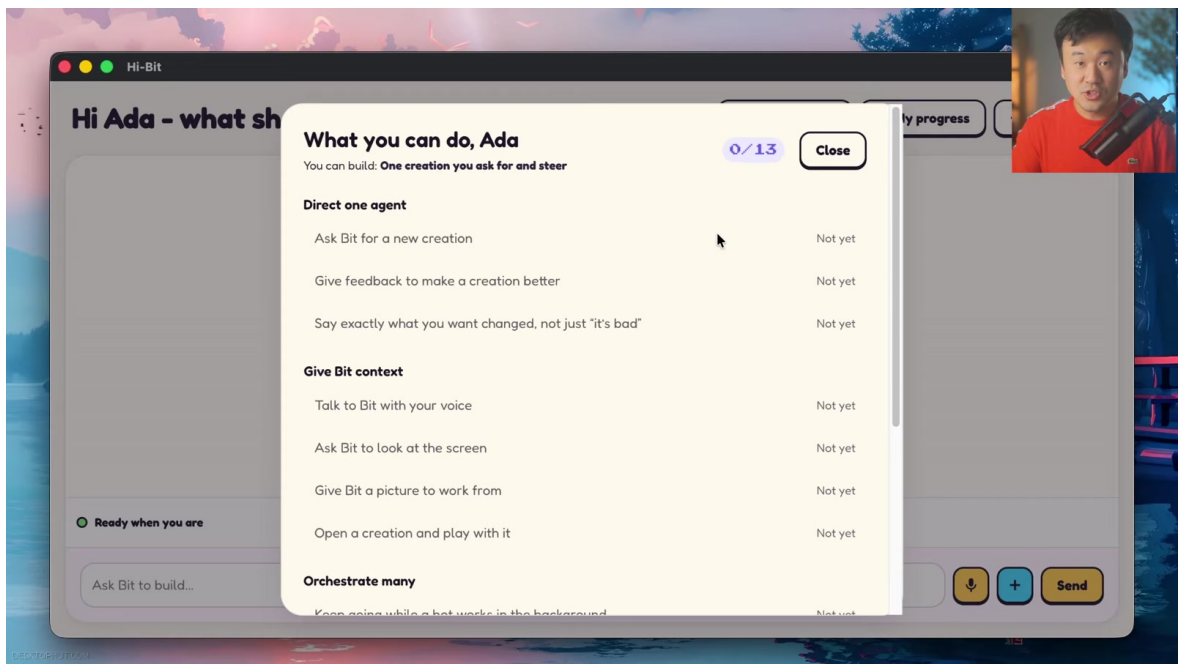


图 13: High Bit 示例: 顶部两个按钮内容相近, 界面也缺少面向儿童的趣味反馈。¹³

¹² 视频画面时间区间: 00:24:20-00:24:38。若画面经过缩放或裁剪, 时间区间仍对应原始本地视频。

¹³ 视频画面时间区间: 00:24:54-00:25:12。若画面经过缩放或裁剪, 时间区间仍对应原始本地视频。

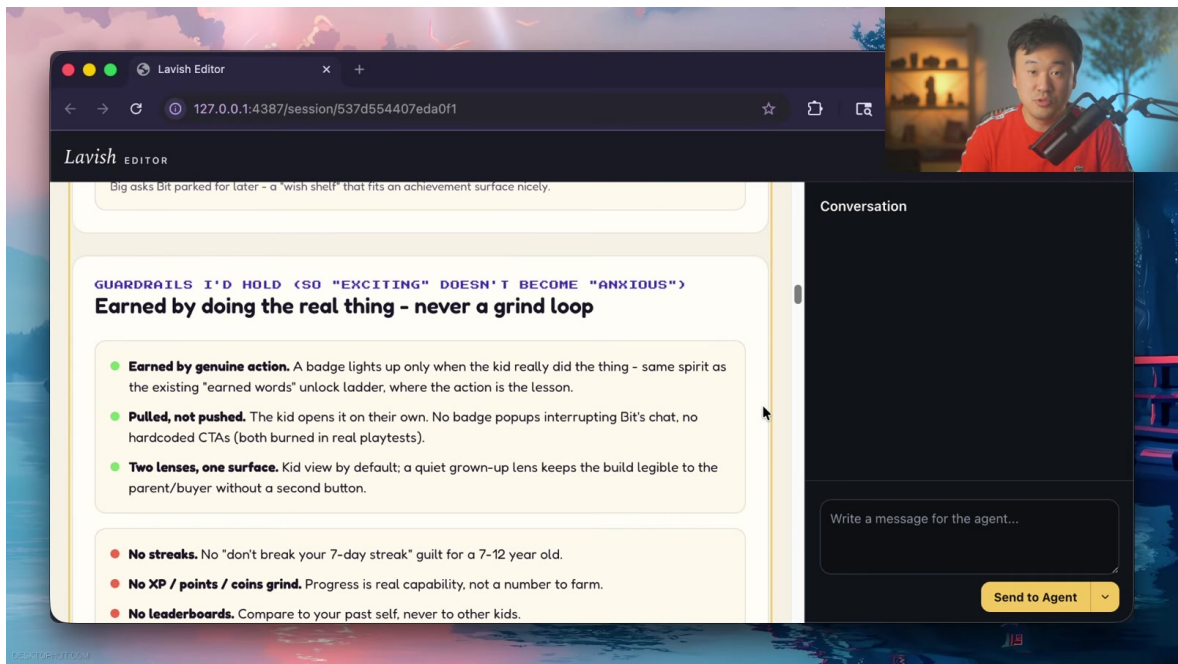


图 14: Lavish editor 把 agent 规划渲染成项目风格一致的页面，便于观察不同方案的实际视觉效果。¹⁴

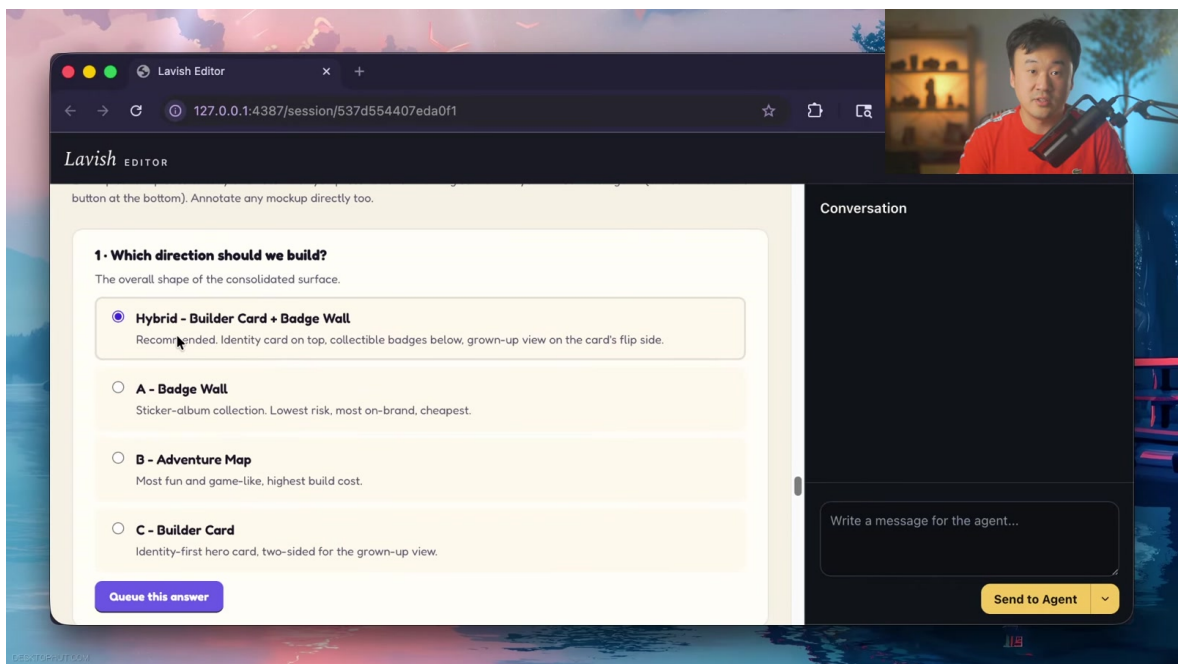


图 15: Lavish 页面底部的决策区：人类可直接选择方向并把反馈送回 agent。¹⁵

规划阶段的质量杠杆

agent 实现阶段越自动化，前置需求歧义越需要被显式压缩。Lavish 的价值体现在提前具体化三个问题：“这个按钮到底长什么样、用户会看到什么、哪里需要改”。这些具体判断能减少后

¹⁴ 视频画面时间区间：00:27:24-00:27:42。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

¹⁵ 视频画面时间区间：00:27:54-00:28:12。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

续实现返工。

4.4 本章小结

与单个船员协作时，效率来自三个环节：语音降低输入成本，AXI 降低工具调用成本，Lavish 降低需求歧义成本。三者都在保护同一个目标：让人类把注意力放在需求和判断上，让 agent 处理实现细节。

5 质量门禁：No Mistakes 把首版代码送到干净 PR

5.1 为什么不能把 diff review 当作主路径

AI 写代码很快。如果每个改动都要求人类逐行 review diff，人类吞吐量会成为硬上限。Kun 的视角更接近工程经理或工程总监：通过流程、文化和验证标准影响质量，减少亲手检查每个中间文件的必要性。No Mistakes 正是在这个位置接管“从 agent 首版实现到可审核 PR”的大量细节。

5.2 No Mistakes 的管线步骤

视频中描述的 No Mistakes 管线包括：在没有分支时创建分支；创建 commit；把验证放进隔离 worktree；分析 agent session 推断真实 intent；在最新 origin/main 上 rebase 并预先处理冲突；启动 fresh context 的 adversarial review；明显问题自动修复，涉及产品判断的歧义升级给人类；按原始 intent 做端到端测试并记录证据；更新相关文档；检查 lint；推送分支并创建 PR；继续 babysit PR，处理后续 CI 失败或合并冲突。

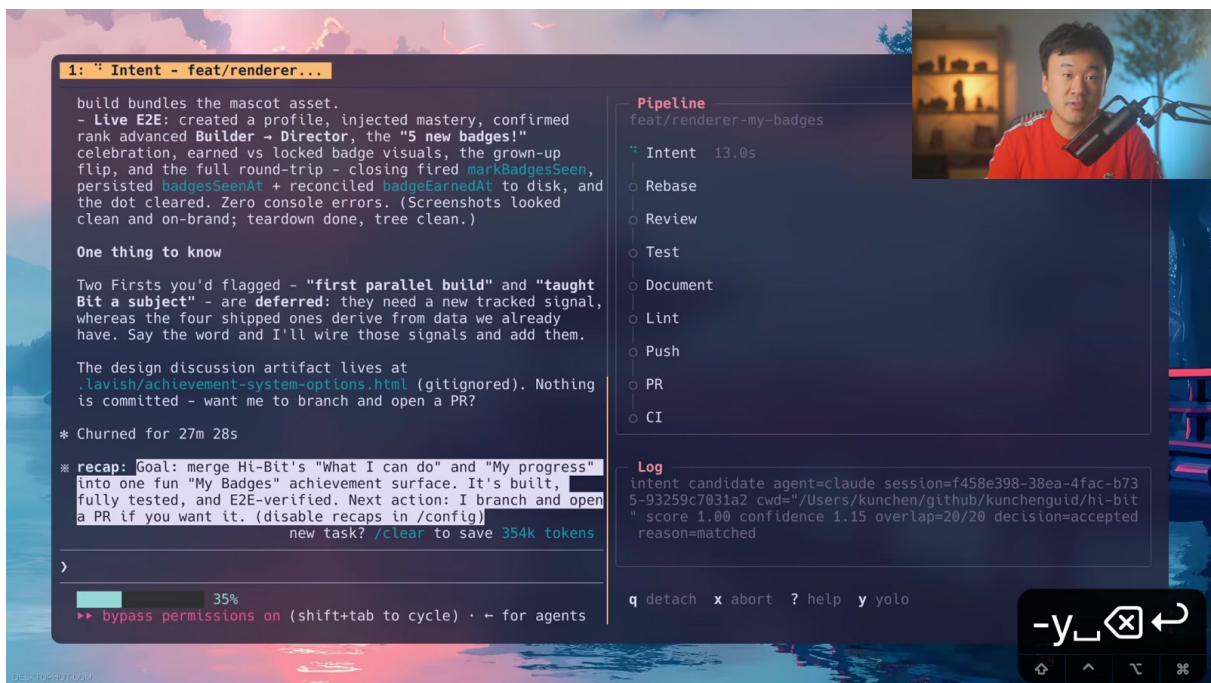


图 16: No Mistakes 在终端中运行：把 agent 生成的首版代码推进到验证管线。¹⁶

¹⁶ 视频画面时间区间：00:30:24-00:30:46。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

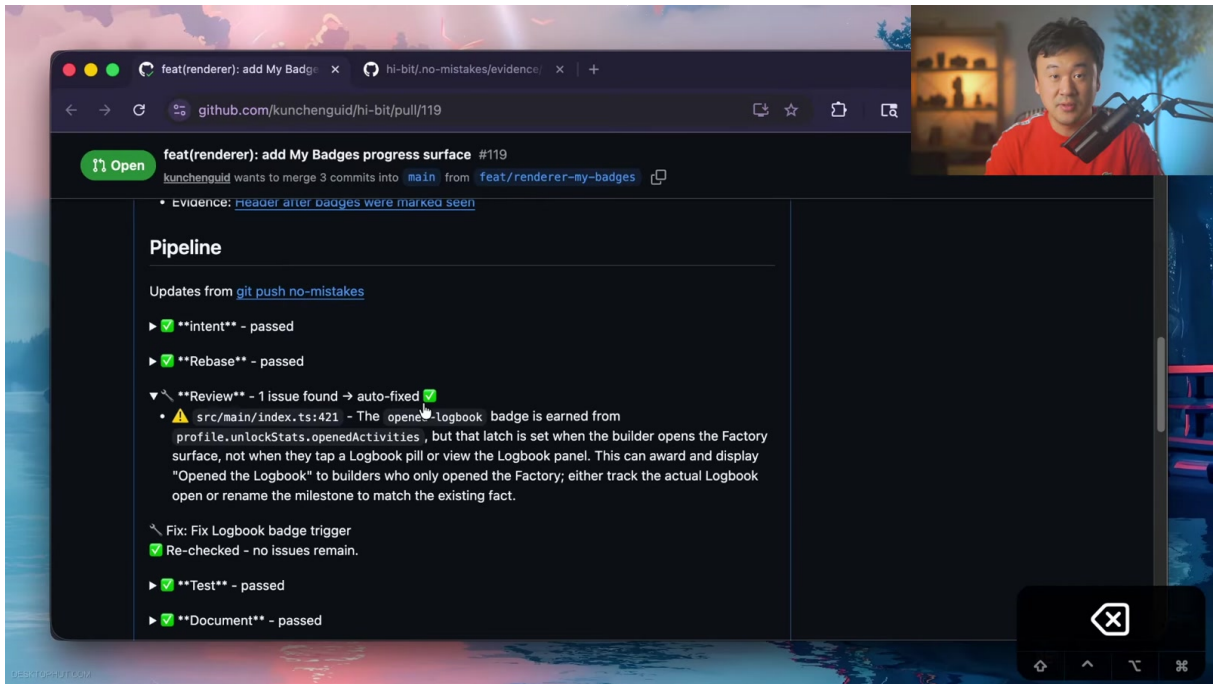


图 17: No Mistakes 生成的 PR：包含原始意图、改动摘要、测试证据、风险评估和管线记录。¹⁷

证据优先于自信描述

PR 中的“测试证据”可以是截图、视频、日志或其他 artifact。它的作用是让人类快速看到“用户会实际经历什么”。对于 UI、端到端流程和产品行为，证据比抽象的“已测试通过”更能支持决策。

5.3 人类时间集中在开头和结尾

视频中的核心节奏是：开头用 Lavish 澄清需求，结尾用 No Mistakes 提供的 PR、风险评估和证据做判断，中间实现阶段交给 agent 和管线。这个节奏可以画成“Plan - Implement - Validate”。实现区间越长、越可靠，人类越能并行启动更多任务。

¹⁷ 视频画面时间区间：00:32:24-00:32:48。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

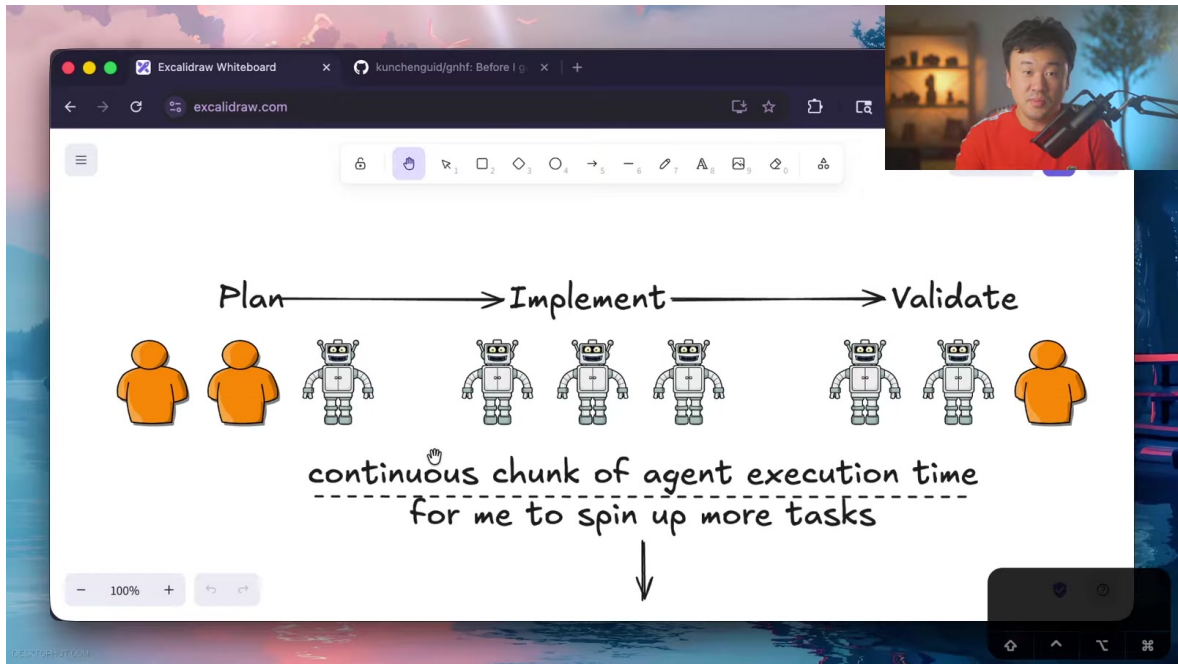


图 18: Plan - Implement - Validate 的手绘图：人类在开头和结尾介入，agent 负责中间连续执行区间。¹⁸

5.4 本章小结

No Mistakes 的价值在于把“我相信 agent 说它做完了”转成“管线给出可检查的 PR、证据和风险分层”。这一步使单个 agent 的产出能进入生产级软件工程流程，也为后续多 agent 并行提供质量边界。

6 延长无人值守时间：Goodnight Have Fun

6.1 为什么需要长循环

一旦人类只在任务开头和结尾介入，一个自然问题就出现了：agent 能否在中间连续工作更久。Kun 用 Goodnight Have Fun 解决长运行场景。它接收一个目标，并在用户设定的 stop condition、token cap 或 iteration cap 内持续执行、修复、再验证。

6.2 适合长循环的目标类型

视频中的示例是在 High Bit 里模拟 7 岁儿童端到端使用产品：跳过家长创建 profile 的步骤，在 app 其余部分尝试不同动作，找到第一个会让儿童困惑或卡住的可用性问題，修掉它，然后 rinse and repeat。讲者还列出更典型的可验证目标：降低页面加载时间、提升 E2E 测试覆盖率、运行假设自动研究来改善某个 metric。

¹⁸视频画面时间区间：00:33:24-00:33:44。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。



图 19: Goodnight Have Fun: 显示循环执行、token 用量、iteration 与 commits, 适合无人值守任务。
19

长循环需要边界

长循环目标必须带有可判断的停止条件, 或者至少有 token 上限、iteration 上限和可回看 commits。缺少边界时, agent 可能把预算耗在低价值探索上。视频中讲者偏好 Goodnight Have Fun, 原因之一是它对 cap 和 stop condition 的控制更细。

6.3 本章小结

Goodnight Have Fun 把 agent 从“一次性执行器”扩展成“可监督循环系统”。它最适合有明确度量或可观察产品行为的任务, 尤其适合夜间或长时间空档。

7 多船员并行: git worktree 与 Treehouse

7.1 并行时的文件系统冲突

当一个 agent 正在原 repo 里执行长循环, 再启动另一个 agent 修改同一目录, 两个会话会互相踩文件、分支、测试产物和临时状态。git worktree 是默认解决方案: 为同一个 repo 创建独立工作目录, 让不同 agent 在不同目录中工作。

但是, 裸用 worktree 会制造管理债。人需要记住每个目录对应什么任务、agent 是否还在运行、什么时候可以删除、怎样清理。视频中讲者用 git worktree add 创建一个随手命名的目录, 随后指出这种目录很容易成为人脑负担。

¹⁹ 视频画面时间区间: 00:35:24-00:35:48。若画面经过缩放或裁剪, 时间区间仍对应原始本地视频。

7.2 Treehouse: 管理 worktree 的轻量层

Treehouse 把 worktree 的创建、复用、状态查看和清理交给工具。进入 repo 后运行 `treehouse`，它把用户放进一个 fresh worktree；再次运行可得到更多 worktree；`treehouse status` 会显示哪些 worktree 在使用，哪些 idle；关闭 tab 后，Treehouse 能将该 worktree 标记为可复用。

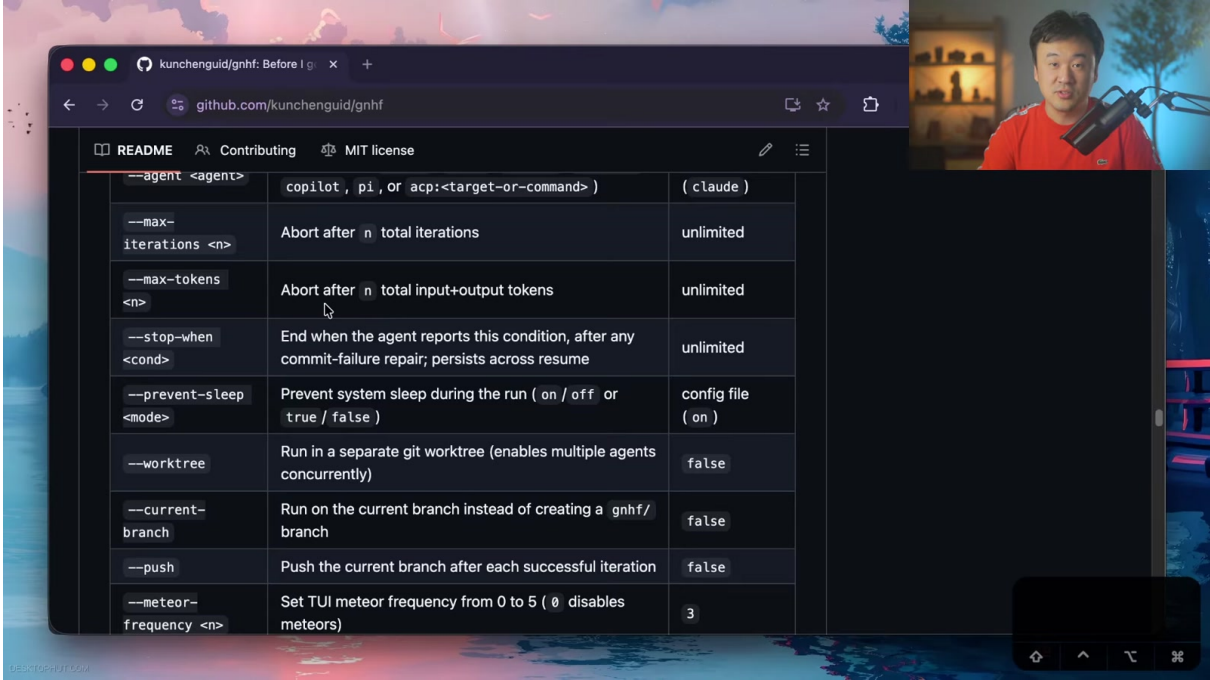


图 20: Treehouse status: 列出多个 worktree、任务说明、状态和路径，降低并行任务的记忆负担。²⁰

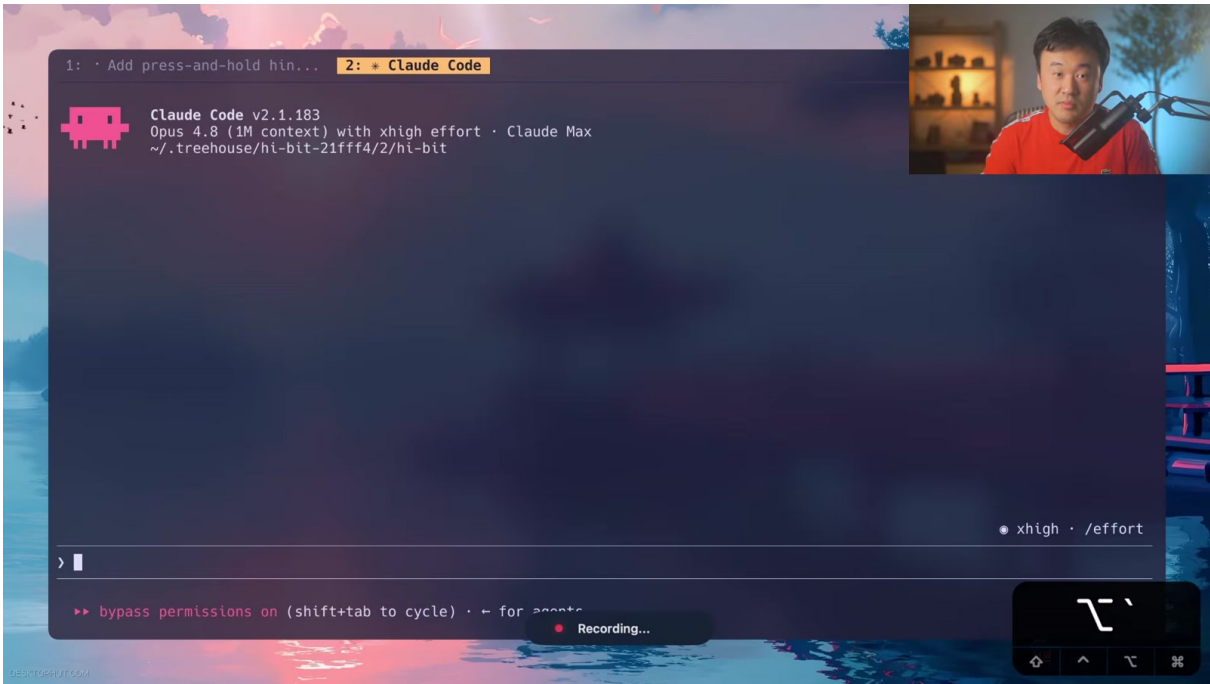


图 21: 多 tab 中运行多个 Treehouse + Claude 会话，每个会话对应独立任务和 worktree。²¹

²⁰ 视频画面时间区间: 00:36:28-00:36:48。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

²¹ 视频画面时间区间: 00:39:22-00:39:42。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

视频中并行启动了三个真实任务：给儿童语音输入 popover 加“可长按说话”的标签；给图片附件下拉菜单增加“截取当前 app 截图作为附件”的动作；修复聊天输入上方 agent status bar 没有持续显示 bot 活动的问题。每个任务都可在自己的 worktree 中运行，必要时继续进入 No Mistakes 管线。

并行的关键约束

并行数量增加后，真正危险的点是人类记不清每个会话的上下文。Treehouse 解决文件系统隔离和 worktree 生命周期，tmux 状态栏帮助观察会话是否等待输入，但它们仍要求人类自己在多个 tab 间切换。这个缺口引出下一章的 First Mate。

7.3 本章小结

Treehouse 让多 agent 并行从“手动创建一堆目录”升级到“可复用、可查看、可回收的 worktree 池”。它解决冲突与清理问题，但还没有解决人的任务调度和上下文切换问题。

8 大副接管调度：First Mate

8.1 从直接管理每个 agent 到管理一个调度者

当多个 agent 会话同时跑起来，人类会遇到新的瓶颈：不断切换 tab、回忆每个任务在做什么、决定谁该继续、谁该验证、谁该开 PR。Kun 把这个瓶颈称为需要“大副”。First Mate 接收更高层的指令，负责拆任务、创建 worktree、启动 agent、跑 No Mistakes，并把需要人类判断的地方抬出来。

8.2 Open Super Whisper 的初始 prompt

视频里 First Mate 使用语音交互时，项目名识别很准。原因是 Open Super Whisper 的 transcription menu 支持 initial prompt，可以把常用项目名和术语放进去，例如 Lavish AXI、GitHub AXI、Chrome DevTools AXI 等。这是一个小细节，却能显著降低语音输入里的专有名词错误。

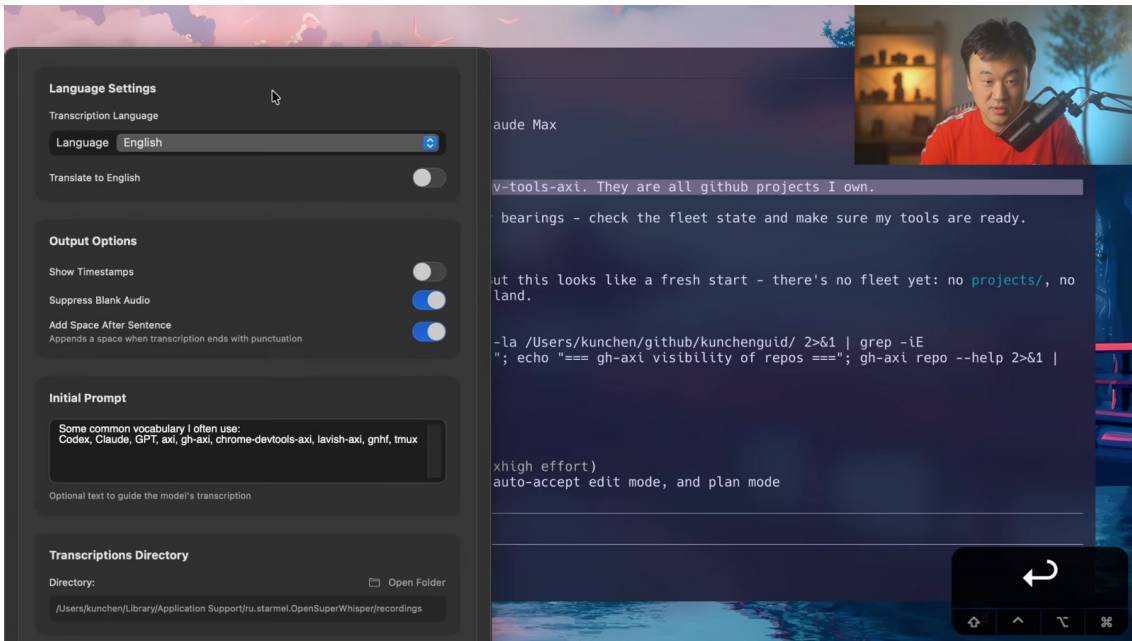


图 22: Open Super Whisper 的 initial prompt: 把常用项目名放进转写提示, 提高专有名词识别。²²

8.3 First Mate 的实际调度过程

示例任务是: 给 Lavish AXI、GitHub AXI、Chrome DevTools AXI 三个项目都加一个 CLI update 命令, 用来把版本更新到 npm 上的最新版本。First Mate 识别出其中包含三个可并行任务; 它在 tmux 中打开多个 tab, 调用 Treehouse 创建 worktree, 运行 agent 完成实现, 再运行 No Mistakes 让 PR 进入审核状态。

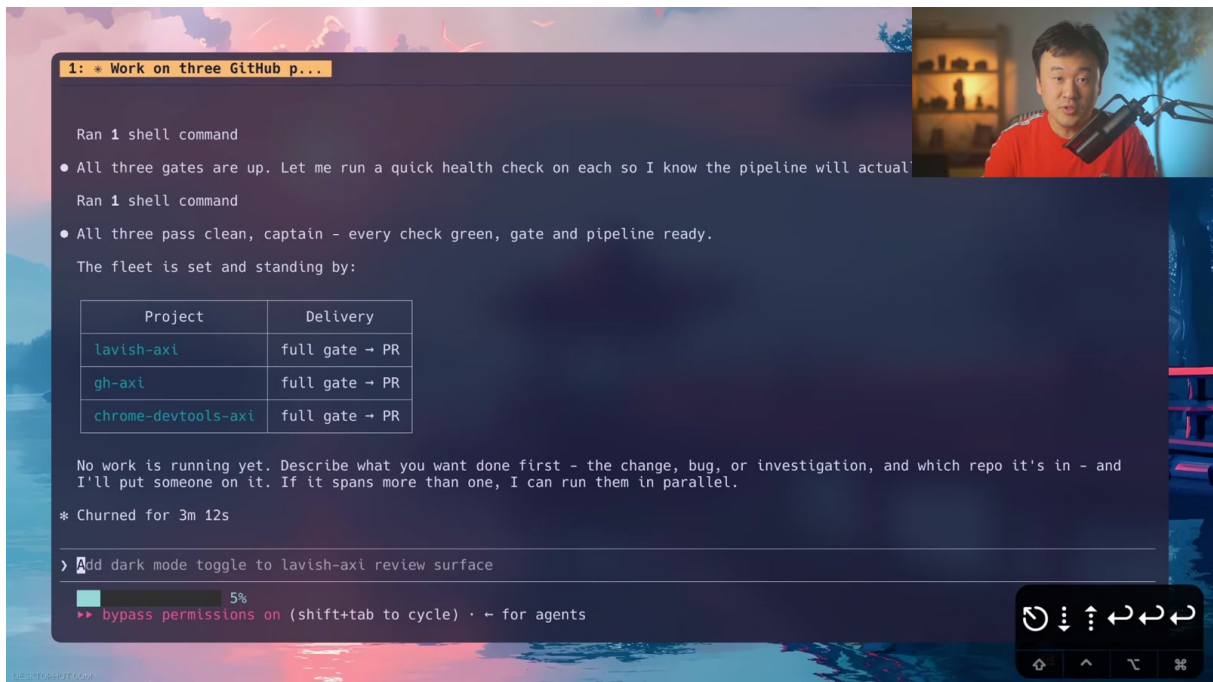


图 23: First Mate 设置 full gates to PR: 用 No Mistakes 作为每个后台任务的质量门禁。²³

²² 视频画面时间区间: 00:41:54-00:42:12。若画面经过缩放或裁剪, 时间区间仍对应原始本地视频。

随后，Kun 又让 First Mate 查看 Lavish AXI 最近三个 open issues，并讨论哪些 actionable。First Mate 在等待后台三个任务时继续拉取 issue，选出最清晰的第 87 号 issue，随后又接收 “address number 87” 的指令。这展示了 First Mate 的关键角色：在人类继续给方向时，它能同时管理多个后台 agent 的进度。

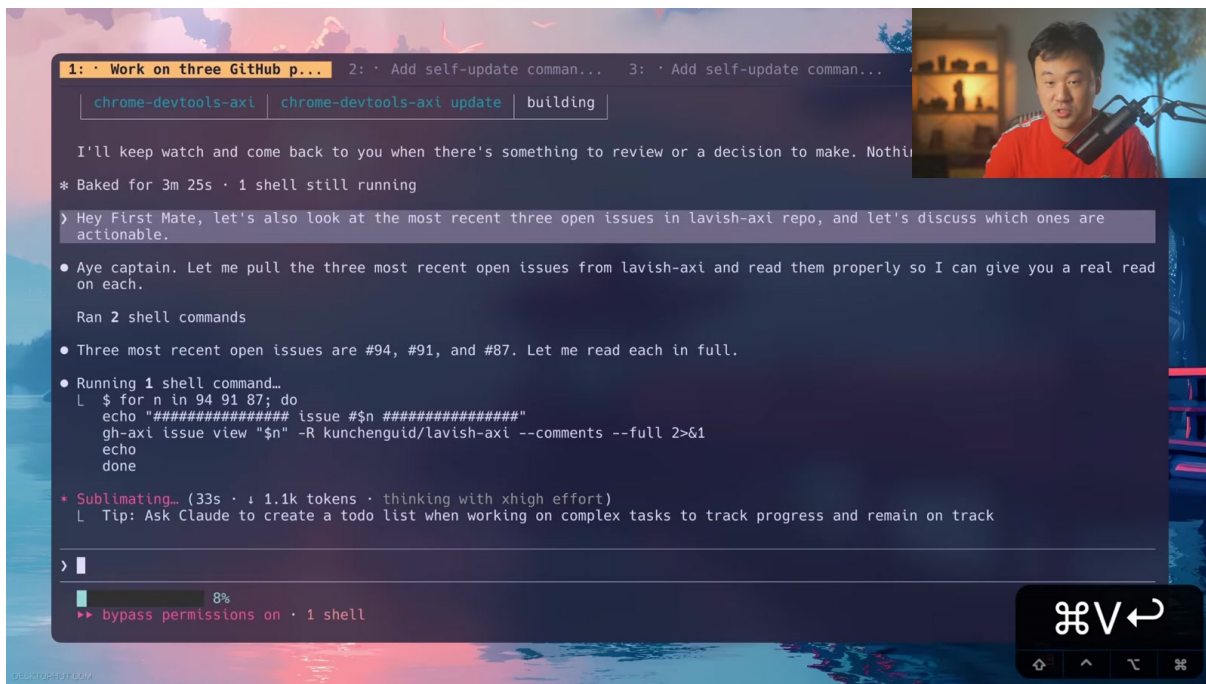


图 24: First Mate 处理 issue 选择与新任务派发：后台 PR 任务仍在进行，前台继续讨论下一项工作。
24

8.4 瓶颈转移：从执行不足到方向不足

First Mate 接管调度后，人类开始“跑不过自己的想法”。这是一种瓶颈转移：实现、验证和并行管理被工具吞掉后，人类需要更多时间去理解用户、研究竞争格局、定义优先级、制作更好的“藏宝图”。视频最后强调，船长需要减少围着每个船员做 whack-a-mole，把注意力放到船队去哪里。

8.5 本章小结

First Mate 把前面所有工具串成一个统一界面：语音输入给方向，Treehouse 负责隔离，agent 负责实现，No Mistakes 负责验证，tmux 负责承载多个会话。人的角色上移到产品方向、优先级和最终判断。

9 实践清单：把视频 workflow 落地到自己的项目

9.1 第一周：先搭一个可恢复的控制台

1. 选择一个稳定终端模拟器，并把字体、主题、快捷键和启动 shell 写进配置文件。
2. 配置 tmux session、pane、window 和状态栏，确保 detach/attach 能恢复工作状态。

²³ 视频画面时间区间：00:42:22–00:42:44。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

²⁴ 视频画面时间区间：00:43:24–00:43:48。若画面经过缩放或裁剪，时间区间仍对应原始本地视频。

3. 选择一个主编辑器，并把搜索、文件跳转、常用修改动作绑定到键盘路径。
4. 只选择一个 agent harness 做主力，保持后续工具链对 harness 低耦合。

9.2 第二周：建立记忆与 skill 纪律

1. 全局 memory file 控制在几十行，写入跨项目偏好和稳定工程原则。
2. 每个 repo 建项目级 memory file，记录目录、术语、测试方式和踩坑经验。
3. 每次 agent 犯重复错，让它把经验写回项目 memory。
4. 把长篇、条件性说明抽成 skill，例如 E2E 测试、发布流程、设计系统使用规则。
5. 安装外部 skill 前做代码审查，检查它会让 agent 执行哪些命令、读取哪些路径、访问哪些网络端点。

9.3 第三周：把需求和验证管线前后夹住

1. 对 UI、产品流程和多方案设计，要求 agent 先生成可视化 artifact，再进入实现。
2. 为每个 PR 定义“可接受证据”：截图、录像、日志、E2E 报告或性能数据。
3. 建立 No Mistakes 类型管线：隔离 worktree、intent 还原、rebase、对抗审查、E2E、文档、lint、PR babysitting。
4. 让人类审核集中在风险评估、证据和产品判断上，逐步减少低风险 diff 的逐行检查。

9.4 第四周：开始并行与调度

1. 学会 git worktree 的基本命令，再用 Treehouse 类型工具管理 worktree 池。
2. 从两个并行 agent 开始，给每个任务明确目标、验证方式和退出条件。
3. 对可验证目标使用长循环工具，例如性能优化、E2E 覆盖率、可用性问题搜寻。
4. 当 tab 切换和任务记忆成为负担时，引入 First Mate 类型调度层。

9.5 本章小结

落地顺序很重要：先控制台，再记忆与 skill，再规划与验证，最后并行调度。跳过前面几层直接上多 agent 并行，往往会把文件冲突、上下文切换和低质量 PR 一起放大。

10 总结与延伸

10.1 讲者的收束观点

视频结尾把整套 journey 收束成“从水手到船长”。起点是没有船，没有船员，只有人类自己写代码；中间逐步拥有终端控制台、agent 船员、memory 和 skill、Lavish 规划、No Mistakes 质量门禁、Goodnight Have Fun 长循环、Treehouse 并行 worktree；终点是 First Mate 帮人类调度一个更大的 crew。讲者把自己的工具开放在 GitHub，希望更多人以有效且愉快的方式学习 agentic engineering。

10.2 本笔记的压缩结论

可以把这节课压缩为五条工程原则：

1. **把上下文写进系统**：memory file 和 skill 让经验跨会话积累，减少每次重新解释项目的成本。
2. **把需求可视化**：复杂产品问题先进入 Lavish 这样的 artifact 讨论，避免文本计划里的歧义滚入代码。
3. **把质量流程化**：No Mistakes 让首版实现经过 intent、rebase、审查、E2E、文档、lint、PR 的固定路线。
4. **把执行拉长**：Goodnight Have Fun 让 agent 在清晰边界内持续发现、修复、验证。
5. **把并行交给调度层**：Treehouse 管 worktree，First Mate 管多个任务和多个 agent，人类负责方向。

10.3 下一步问题

这套 workflow 真正考验人的地方会转到更上游：用户研究怎样变成高质量任务；竞争格局怎样变成优先级；产品愿景怎样拆成可验证、可并行、可回滚的小步；哪些任务允许长循环，哪些必须由人类即时判断。换句话说，agentic engineering 的成熟标志不在于启动了多少 agent，而在于每个 agent 的任务都能被清晰定位在产品方向、验证证据和风险边界之内。

最后的可执行要点

在自己的项目里先选一个小功能做完整演练：用语音描述目标，用可视化 artifact 澄清需求，让 agent 实现，交给验证管线生成 PR 和证据，再用 worktree 跑第二个小任务。一次跑通后，再扩大并行度。

10.4 本章小结

Kun 的 workflow 展示的是一套生产级个人工程系统：终端提供控制台，memory 与 skill 提供组织记忆，AXI 与 Lavish 提供 agent 友好的工具接口，No Mistakes 提供质量门禁，Goodnight Have Fun 提供长循环，Treehouse 与 First Mate 提供并行调度。人类的核心职责随之上移到定义目标、理解用户、安排优先级和做最终产品判断。